AN INTELLIGENT CLOUD-BASED ACADEMIC RESOURCE SHARING SYSTEM CLOUD- NATIVE MICROSERVICE ARCHITECTURE

CLOUD COMPUTING

Submitted by

DINISHA R (230701080)
ENIYA B A (230701085)
JAYAPRADHA P (230701130)

in partial fulfilment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

RAJALAKSHMI ENGINEERING COLLEGE

NOVEMBER 2025

ANNA UNIVERSITY, CHENNAI

BONAFIDE CERTIFICATE

Certified that this project report titled **"ACADEX – A SYSTEM FOR DIGITAL ACADEMIC RESOURCE STORAGE AND INTELLIGENT INTERACTION"** is the bonafide work of **DINISHA R (230701080), ENIYA B A (230701085), JAYAPRADHA P (230701130)** who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Dr. E. M Malathy
Professor &
Head of The Department
Department of Computer Science and Engineering
Rajalakshmi Engineering College

SIGNATURE

Dr.M.Ayyadurai
Supervisor
Associate Professor
Department of Computer Science and Engineering
Rajalakshmi Engineering College

Submitted to Project Viva Voce Examination held on_____

Internal Examiner **External Examiner**

ACKNOWLEDGEMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavor to put forth this report. Our sincere thanks to our Chairman **Mr. S.MEGANATHAN, B.E, F.I.E.**, our Vice Chairman **Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S.**, and our respected Chairperson **Dr. (Mrs.) THANGAM MEGANATHAN, Ph.D.,** for providing us with the requisite infrastructure and sincere endeavoring in educating us in their premier institution.

Our sincere thanks to **Dr. S.N. MURUGESAN, M.E., Ph.D.**, our beloved Principal for his kind support and facilities provided to complete our work in time. We express our sincere thanks to **Dr.MALATHY E.M, Ph.D.**, Professor and Head of the Department of Computer Science and Engineering for her guidance and encouragement throughout the project work. We convey our sincere and deepest gratitude to our internal guide, **DR. M. AYYADURAI M.E.,Ph.D.,** Associate Professor, Department of Computer Science and Engineering for his valuable guidance throughout the course of the project.

DINISHA R
ENIYA B A
JAYAPRADHA P

ABSTRACT

College students increasingly rely on digital platforms for storing, managing, and sharing academic resources such as notes, assignments, and project files. However, many existing systems lack centralized access, intelligent assistance, and secure file management, leading to inefficiencies in organizing and retrieving important academic content. Students often struggle with scattered data across devices, lack of structured storage, and absence of smart tools to assist in understanding or utilizing their materials effectively. The Acadex platform aims to provide a unified, intelligent academic resource management system that enables students to upload, store, and access their files securely in a centralized environment. Unlike traditional storage systems, Acadex integrates cloud-based file management with Generative AI capabilities to enhance user interaction and productivity. The platform allows users to upload files, view and manage stored content, and interact with an AI-powered assistant for queries, explanations, and recommendations related to their academic materials. Security and scalability are key aspects of the system. The platform leverages cloud services for reliable backend hosting and storage, ensuring efficient handling of user requests and data. Authentication mechanisms are implemented to provide secure access, while containerization techniques ensure consistent deployment across environments. The integration of AI services enables intelligent responses, improving user experience by providing contextual assistance and insights. The system is designed using a modular architecture, where the frontend provides an intuitive user interface, and the backend manages file operations, API handling, and AI integration. Cloud-based storage ensures that uploaded files are securely maintained and easily accessible, while REST APIs facilitate seamless communication between components. The expected outcome of the Acadex platform is to improve academic productivity by providing a structured and intelligent system for managing digital resources. By combining cloud storage with AI-driven assistance, the platform reduces time spent on searching and understanding materials, enhances accessibility.

TABLE OF CONTENTS

CHAPTER 1 - INTRODUCTION

1.1 PROBLEM STATEMENT

In the modern academic environment, students increasingly depend on digital tools to manage their study materials, including notes, assignments, and project files. However, the current systems used for managing academic resources are often fragmented, inefficient, and lack intelligent support. Students typically store files across multiple platforms such as local storage, cloud drives, and messaging applications, leading to difficulties in organization, retrieval, and accessibility.

One of the major challenges faced by students is the absence of a centralized platform for managing academic content. Important files are often misplaced, duplicated, or difficult to locate, resulting in wasted time and reduced productivity. Additionally, traditional storage solutions provide only basic file management capabilities and lack advanced features such as intelligent search, contextual assistance, and automated insights.

Another critical issue is the lack of intelligent interaction with stored data. Students frequently require explanations, summaries, or guidance related to their academic materials, but existing systems do not provide AI-driven assistance. This forces students to rely on external tools, increasing complexity and reducing efficiency.

Security and reliability also remain concerns. Many students store sensitive academic data without proper access control or secure storage mechanisms, which can lead to data loss or unauthorized access. Furthermore, existing platforms do not offer seamless integration between storage, processing, and intelligent services.

These challenges highlight the need for a unified, secure, and intelligent academic resource management system that not only stores files but also enhances their usability through AI-driven features and cloud-based scalability.

1.2 OBJECTIVE OF THE PROJECT

The primary objective of the Acadex project is to design and develop an intelligent academic resource management platform that enables students to securely upload, store, manage, and interact with their academic files in a centralized environment. The system integrates cloud computing and Generative AI to enhance productivity, accessibility, and user experience.

1. **Centralized File Management:**
   Provide a unified platform where students can upload, store, and access all academic resources in one place, eliminating fragmentation and improving organization.

2. **Intelligent AI Assistance:**
   Integrate Generative AI capabilities to enable chatbot functionality, allowing users to ask questions, receive explanations, and get recommendations related to their academic content.

3. **Secure and Reliable Storage:**
   Ensure safe storage of files using cloud-based services with proper access control mechanisms, reducing the risk of data loss and unauthorized access.

4. **Scalability and Performance:**
   Design the system using cloud infrastructure and containerization to handle increasing user demand efficiently.

5. **Improved Productivity:**
   Reduce time spent searching and understanding academic materials by providing intelligent assistance and structured storage.

1.3 SCOPE AND BOUNDARIES

The scope of this project includes the development and deployment of a full-stack web application integrated with cloud services and Generative AI capabilities. The system supports essential functionalities such as file upload, storage, retrieval, and AI-based interaction.

The frontend is developed to provide a user-friendly interface for managing files and interacting with the AI assistant. The backend handles API requests, file processing, and communication with cloud services. Azure App Service is used for backend hosting, while Azure Static Web Apps serves the frontend efficiently.

File storage is managed using Azure Blob Storage, ensuring scalable and secure storage of uploaded resources. The system integrates Azure OpenAI Service to provide chatbot functionality, enabling users to interact with their data intelligently.

Containerization using Docker ensures consistent deployment across environments, and Azure Container Registry is used for managing container images. The application follows a modular architecture, allowing easy scalability and future enhancements.

The project is limited to academic file management and AI-based interaction. Features such as advanced analytics, multi-institution support, and collaborative editing are considered outside the current scope but can be included in future development.

1.4 STAKEHOLDERS AND END USERS

Stakeholders: Project Developers / Team Members **-:** Responsible for designing, developing, deploying, and maintaining the system, including backend APIs, frontend interface, and AI integration.

End Users:

1. **Students:**
   Primary users who upload, store, and manage academic files and interact with the AI assistant.
2. **Learners Seeking Assistance:**
   Students who use the chatbot feature to understand concepts, get explanations, and improve learning efficiency.
3. **New Users / Beginners:**
   Individuals who benefit from organized storage and intelligent guidance for academic materials.

1.5 TECHNOLOGIES USED

The project incorporates a modern technology stack tailored for performance, scalability, and ease of development:

Table 1. Frontend Technologies

React	For building frontend user interface of marketplace
JavaScript	Core programming language for frontend.
CSS	Styling and layout design.

Table 2: Backend Technologies

Node.js	Backend runtime environment.
Express.js	Framework for building APIs.
Multer	File upload handling.
Axios	API communication.
Blob Storage	For storing files

Table 3: Cloud and Infrastructure

Microsoft Azure	The primary cloud provider for hosting and managing services.
Azure Container Apps	For container orchestration with auto-scaling.
Azure App Service	For Backend hosting.
Azure Storage Account	For blob storage of user-uploaded files.
Azure Container Registry	Stores Docker images
Terraform	For IaaC provision.
Docker	For containerizing applications

Table 4: AI and Machine Learning

Azure OpenAI Service	Provides GPT-based AI chatbot functionality.

Table 5: DevOps and CI/CD

GitHub Actions	For automating workflows, builds, and deployments.
Docker Hub	As a registry for storing and distributing container images.
Azure CI/CD	For Automated deployment pipelines.

1.6 ORGANIZATION OF THE REPORT

This report is structured to provide a comprehensive overview of the project development process.

- **Chapter 1** introduces the problem statement, objectives, scope, stakeholders, and technologies used.
- **Chapter 2** describes the system architecture and design, including workflow and component interaction.
- **Chapter 3** explains the implementation details, including backend development, frontend design, and AI integration.
- **Chapter 4** focuses on deployment, cloud services, containerization, and monitoring.
- **Chapter 5** presents results, performance evaluation, and challenges faced during development.
- **Chapter 6** concludes the project with outcomes and future enhancements.

CHAPTER 2 - SYSTEM DESIGN AND ARCHITECTURE

2.1 REQUIREMENT SUMMARY

Functional Requirements

The Acadex system is designed as a cloud-based platform that integrates file management with Generative AI capabilities to enhance student productivity. The system is deployed using Microsoft Azure services to ensure scalability, reliability, and performance.

The frontend of the application is developed using React and hosted on Azure Static Web Apps. This service provides fast and secure hosting of static content with global accessibility and minimal cost. It ensures seamless user interaction for uploading, viewing, and managing academic files.

The backend is developed using Node.js and Express.js, handling API requests such as file upload, file retrieval, and AI-based interactions. The backend is hosted on Azure App Service, which provides a scalable and managed environment for running server-side logic.

For file storage, Azure Blob Storage is used to store uploaded files such as notes, documents, and academic resources. This ensures secure, scalable, and highly available storage.

The system integrates Azure OpenAI Service to provide Generative AI capabilities. This allows users to interact with an AI assistant for tasks such as explaining concepts, answering queries, and suggesting improvements.

Containerization is implemented using Docker to ensure consistent deployment across environments. Azure Container Registry is used to store Docker images for deployment purposes.

REST APIs are used for communication between frontend, backend, and AI services, ensuring seamless data flow across the system.

Non-Functional Requirements

The Acadex platform is designed to meet key non-functional requirements such as performance, scalability, security, and reliability.

To ensure high performance, the system targets API response times below 300ms and page load times under 3 seconds. Azure Static Web Apps ensures fast frontend delivery using optimized content distribution.

The system is designed to support multiple concurrent users through scalable backend services. Azure App Service enables automatic scaling based on demand, ensuring smooth performance during peak usage.

Availability is maintained through Azure's cloud infrastructure, providing high uptime and reliable service access.

Security is ensured through authentication mechanisms and secure handling of API keys. Sensitive data such as API keys can be managed using secure practices and cloud-based services.

Data durability is achieved through Azure Blob Storage, which ensures reliable and persistent storage of uploaded files.

The system also focuses on maintainability and modular design. The architecture separates frontend, backend, and AI services, making it easier to update and extend functionality.

2.2 PROPOSED SOLUTION OVERVIEWS

The proposed Acadex system is designed as a cloud-native application that seamlessly brings together the frontend, backend, storage, and AI components into a single, well-integrated platform. The frontend, developed using React, acts as the user-facing layer where students can easily upload, view, and manage their academic files while also interacting with an AI assistant. It provides a smooth and responsive interface, ensuring that users can perform actions without complexity.

Behind the scenes, the backend built with Node.js and Express.js functions as the core processing unit of the system. It receives requests from the frontend, handles file uploads, retrieves stored data, and manages communication with external services. When a user uploads a file, the backend processes the request, ensures the file is properly handled, and stores it securely in Azure Blob Storage, which offers scalable and reliable cloud-based storage. Similarly, when a user asks a question or interacts with the chatbot, the backend forwards the request to Azure OpenAI Service, which generates an intelligent response that is sent back to the user.

The entire system is structured using a modular architecture, meaning each component—frontend, backend, storage, and AI—operates independently but is connected through REST APIs. This separation ensures that each part can be developed, maintained, and scaled without affecting the others. Overall, this architecture provides a flexible, efficient, and scalable solution where data flows smoothly between components, enabling both secure file management and intelligent user interaction within a unified system.

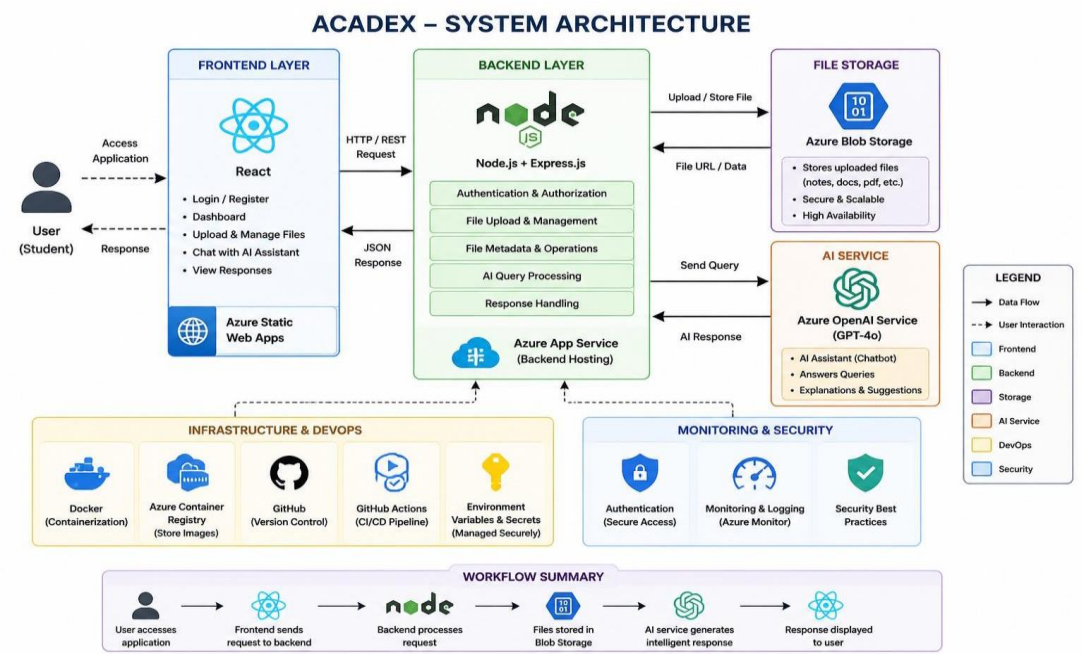

Fig 2.2.1 System Architecture

2.3 CLOUD DEPLOYMENT STATERGY

The Acadex platform is deployed on Microsoft Azure, making use of cloud-native services to ensure that the application is scalable, reliable, and capable of handling real-time user demands. By leveraging Azure, the system avoids the limitations of local infrastructure and benefits from high availability, automatic scaling, and global accessibility.

The frontend of the application is hosted using Azure Static Web Apps, which is specifically optimized for serving static content like HTML, CSS, and JavaScript. This ensures fast loading times and smooth user experience, as the content is delivered through a globally distributed network. Users can access the application quickly from different locations without performance issues.

The backend is deployed on Azure App Service, which provides a managed environment for running Node.js applications. This service handles all server-side operations such as processing requests, managing file uploads, and interacting with AI services. It also supports automatic scaling, meaning the system can handle increased traffic without manual intervention.

For file storage, Azure Blob Storage is used to securely store all uploaded files. It provides high durability and availability, ensuring that user data is safely stored and can be accessed whenever needed. This eliminates the risk of data loss and supports large-scale storage requirements.

To add intelligence to the system, Azure OpenAI Service is integrated. This allows the application to provide chatbot functionality, where users can interact with an AI assistant to get explanations, answers, and suggestions related to their academic content.

Docker is used to containerize the backend application, which means the entire application along with its dependencies is packaged into a container. This ensures that the application behaves consistently across different environments, whether it is development, testing, or production. Azure Container Registry is used to store and manage these Docker images, making deployment more efficient and organized.

Overall, the system follows a layered and modular architecture where the frontend, backend, storage, and AI services are loosely coupled and communicate through APIs. This design allows each component to function independently while still working together seamlessly, providing flexibility for future upgrades and ensuring that the system can scale efficiently as user demand grows.

2.4 INFRASTRUCTURE REQUIREMENTS

Compute Resources:

- **Azure Container Apps:** Instances with 0.25 vCPU and 0.5 GiB memory; auto-scaling from 0 to 10 replicas based on CPU (>60%) or requests per second.
- **Azure App Service:** Used for hosting backend APIs

Storage Requirements:

- **Blob Storage:** Locally redundant storage (LRS) in Hot tier for frequent access, estimated 100 GB initial capacity.

Containerization :

- **Docker :** Used to package backend application.
- **Azure Container Registry :** Used to Store the container images.

AI Integration :

- **Azure OpenAI Services :** Provides chatbot functionality Handles user queries and responses.

2.5 AZURE SERVICES MAPPING AND JUSTIFICATION

Table 6: Service Mapping

Requirement	Azure Service	SKU/Tier	Purpose	Justification	Est. Monthly Cost (₹)
Frontend Hosting	Azure Static Web Apps	Standard	Host user interface	Free hosting, SSL, CDN	Free (Student Credit)
Backend Logic & APIs	Azure App Services	Free F1 / Basic	Host Node.js backend	Scalable, managed environment	Free / ~1000
Media Storage	Azure Blob Storage	Hot Tier, Geo-Redundant Storage	Store uploaded files	Secure, scalable, cost-efficient	300
AI Integration	Azure OpenAI Service	Pay-per-use	Chatbot functionality	Intelligent AI responses	50–100
Monitoring & Logs	Azure Monitor + Application Insights	Standard	Log and track API health	Proactive maintenance	400
Image Storage	Azure Container Registry	Basic	Store Docker images	Supports container deployment & versioning	300
CI/CD Automation	Github Actions	Basic	Automated builds & deployments	Seamless GitHub integration	Free (Student Credit)

CHAPTER 3 - DEVOPS IMPLEMENTATION

3.1 CONTINUOUS INTEGRATION AND DEPLOYMENT STATERGY

The project uses a CI/CD pipeline built with GitHub Actions to automate the build and deployment process. This setup ensures that every code update is automatically tested, built, and deployed to Azure services without manual intervention.

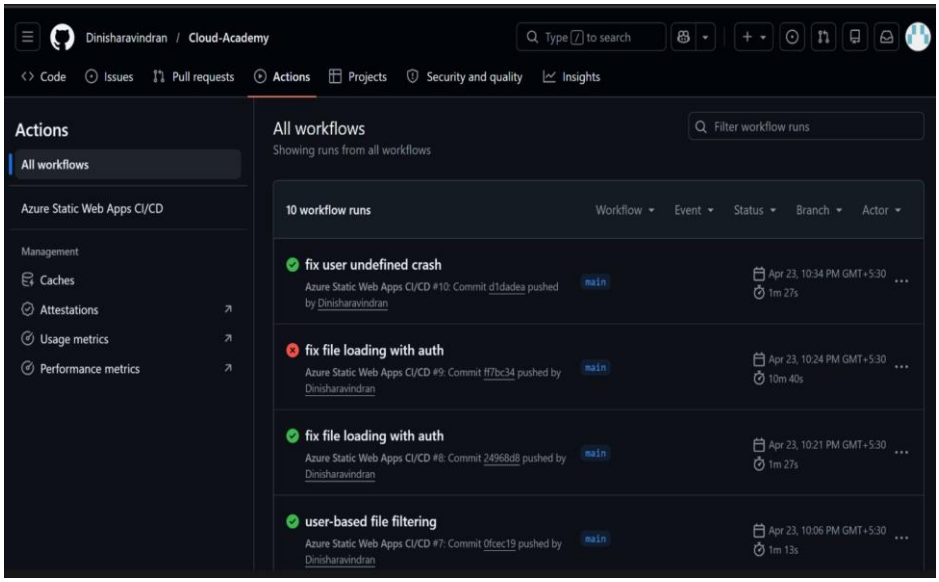

Figure 1. Github Actions Workflow

The pipeline follows a 2-stage deployment pattern:

Backend: Builds and deploys the backend application using Docker and Azure Container Registry.
Frontend: Builds and deploys the React application to Azure Static Web Apps.
The workflow is triggered when code is pushed to the main branch or when pull requests are created or updated.

Pipeline Stages in Detail:

Stage 1: Backend Deployment (build-and-deploy-backend)

1. **Versioning and Containers:** A unique version tag is created for the container image using the date and github commit ID (e.g., v20251029-a1b2c3d). The image is then built and pushed to the registry.

2. **Infrastructure as Code (IaC):** Terraform manages all Azure resources. It ensures that the infrastructure is consistent and changes are applied.

3. **Azure Deployment:** Secure access to Azure is handled through a Service Principal using secrets stored in GitHub. The new container image is deployed to Azure Container Apps using a zero-downtime rolling update.

4. **Health Check:** After deployment, the pipeline waits and performs an automated check against the backend's /healthendpoint to confirm the service is running correctly before moving on.

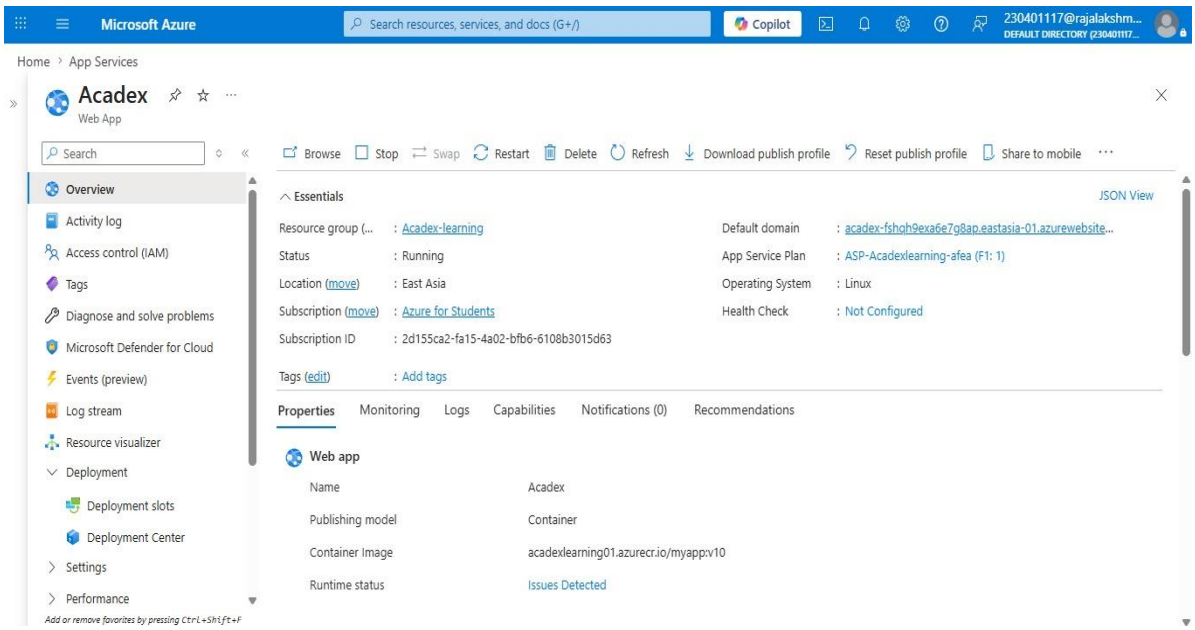

Figure 2. Backend Deployment Job Run

Stage 2: Frontend Deployment (build-and-deploy-frontend)

1. **Dynamic Configuration:** The backend URL is passed to the frontend during build time.

2. **Build Process:** The React application is built using Vite and optimized into static files.

3. **Hosting:** The frontend is deployed to Azure Static Web Apps for fast and secure hosting**.**

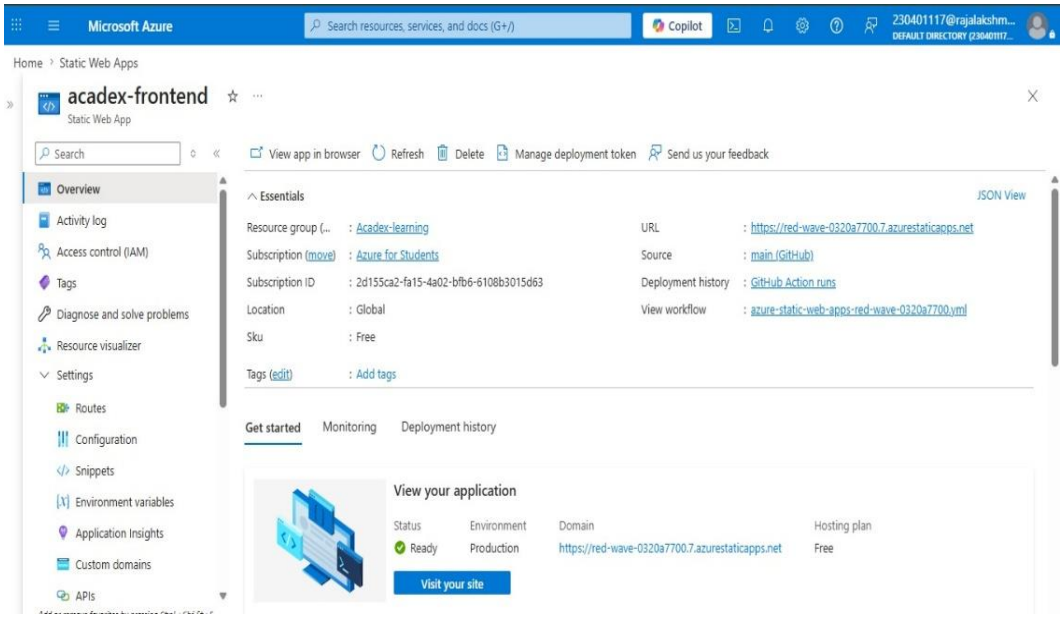

Figure 3. Frontend Deployment Job Run

Required GitHub Secrets

Table 7. List of Github Secrets

1	AZURE_CLIENT_ID
2	AZURE_CLIENT_SECRET
3	AZURE_STATIC_WEB_APPS_API_TOKEN
4	AZURE_STORAGE_ACCOUNT_NAME
5	AZURE_SUBSCRIPTION_ID
6	AZURE_TENANT_ID
7	CONTENT_SAFETY_KEY
8	DB_PASSWORD
9	DOCKERHUB_TOKEN
10	DOCKERHUB_USERNAME
11	OPENAI_API_KEY

3.2 TERRAFORM INFRASTRUCTURE-AS-CODE (IaC)

The Terraform code is organized using a standard three-file structure:

main.tf: Contains the primary definitions for all cloud resources.

variables.tf: Holds input values and settings, like resource names or location.

outputs.tf: Defines data outputs, such as resource URLs or hostnames, for use by other systems (like the CI/CD pipeline).

Core Infrastructure Components

The IaC setup provisions and manages all required Azure services within a centralized Resource Group, ensuring consistency and easier management.

Backend and Container Orchestration

Container App Environment: Provisions the managed hosting environment using Azure App Service, which acts as the runtime platform for the backend application. This environment handles auto-scaling, request processing, and networking without requiring manual infrastructure management.

Container App: Defines the backend application deployed as a Docker container. The container image is built during the CI/CD process and stored in Azure Container Registry (ACR), from where it is pulled and deployed to Azure services.

Scaling: The system supports automatic scaling based on traffic. Azure App Service dynamically adjusts compute resources to handle varying workloads, ensuring both performance and cost efficiency.

Configuration: Application configuration details such as API keys, service endpoints, and storage credentials are securely managed using environment variables and GitHub Secrets, avoiding hardcoded sensitive data.

Database

Azure Blob Storage: Provides scalable cloud storage for user-uploaded academic resources such as notes, documents, and files. It ensures high availability and secure access to stored content.

Blob Container: A dedicated container is created to store files, with controlled access permissions. This setup allows efficient file retrieval and integration with the application frontend.

Serverless Functions (AI Services)

AI Integration: The system integrates Azure OpenAI Service to provide intelligent responses and academic assistance.

Implementation: The AI functionality is handled through backend APIs, which process user queries and communicate with the Azure OpenAI service to generate responses.

CI/CD Integration

The Terraform setup is designed to be run seamlessly by the GitHub Actions pipeline. During the CI/CD execution:

1. Required secrets (such as API tokens, storage account details, and container registry credentials) are securely passed from GitHub Secrets into the deployment environment variables.

2. The application's Docker image is built during the pipeline and pushed to Azure Container Registry (ACR), ensuring that the latest version of the application is always available for deployment.

3. Deployment configurations and outputs (such as application URLs and service endpoints) are used during later stages of the pipeline, including frontend build and deployment.

3.3 CONTAINERIZATION STRATERGY (DOCKER)

The project uses a disciplined containerization strategy to package the Go backend application into small, secure, and efficient Docker images. This approach is optimized for fast and reliable deployment on Azure Container Apps.

Image Build Process: Multi-Stage Optimization

A multi-stage Docker build is used to create lightweight and optimized images. This separates the build phase from the runtime environment, improving performance and reducing image size.

Builder Stage: The process starts with a Node.js base image where the application dependencies are installed and the source code is prepared. The application is built and configured within this

stage to ensure all required modules are included.

Runtime Stage: A minimal runtime image is used to run the application. Only the necessary files and dependencies are copied from the builder stage, reducing the overall image size and improving security.

Deployment, Scaling, and Versioning

1. **Azure Container Registry Integration:**

   The Docker image is built and pushed to Azure Container Registry (ACR), which securely stores container images and supports version control.

2. **Deployment:**

   Azure services pull the latest container image from ACR and deploy it to the backend hosting environment.

3. **Scaling:**

   The application automatically scales based on demand using Azure managed services, ensuring efficient resource usage.

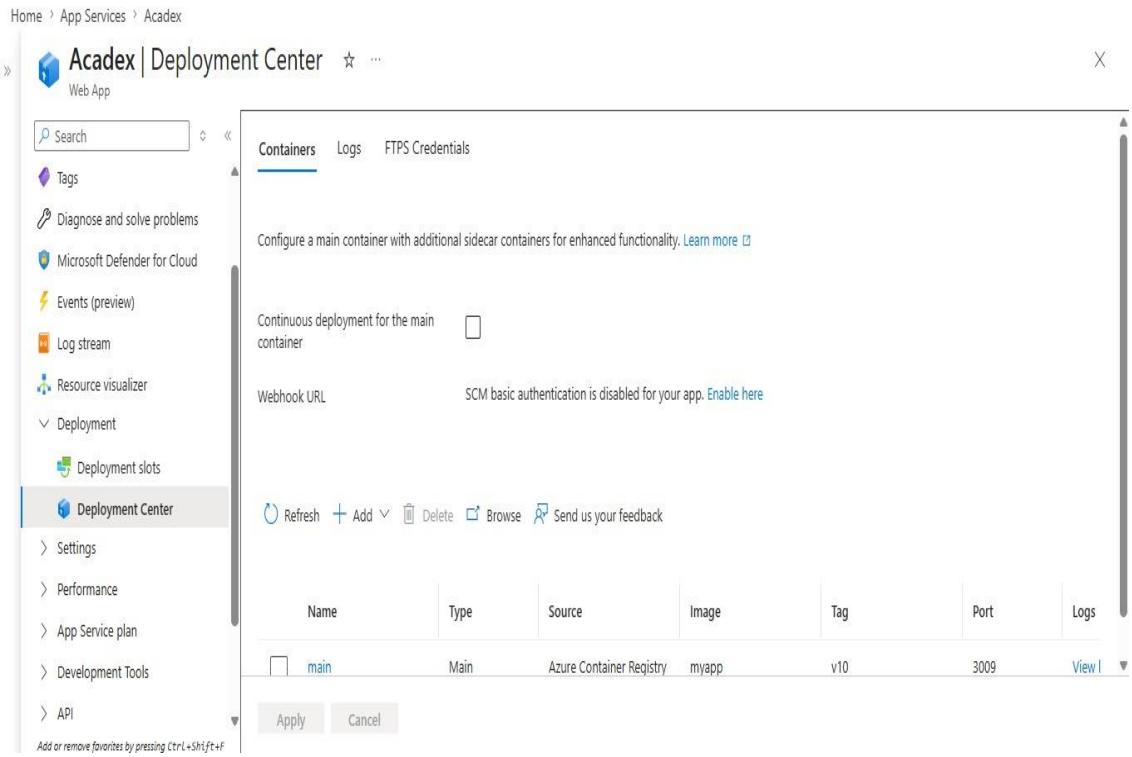

Figure 4. Docker file

3.4 GENAI INTEGRATION AND AZURE AI SERVICE MAPPING

1. AI-Based Academic Assistance (Azure OpenAI Service)

This feature uses Azure OpenAI Service to provide intelligent responses and academic assistance when users interact with the system.

Implementation:

The integration is handled through backend APIs, which act as an interface between the frontend application and the Azure OpenAI service. This approach allows the AI processing to be managed efficiently along with the application logic.

Process:

The system receives user queries from the frontend interface.

The backend processes the input and sends it to the Azure OpenAI Service.

A structured prompt is generated based on the user request, ensuring that the response is relevant and context-aware. The AI service then processes the request and returns an appropriate response.

2.Content Safety Moderation (Azure Content Safety API)

This service is integrated to ensure that all user inputs and uploaded content follow safety guidelines before being processed or displayed in the system.

Implementation: Similar to the AI service, this module is used to analyze user content rather than generate it. It acts as a filtering layer to detect unsafe or inappropriate inputs.

Process: Every user input or uploaded data is analyzed by the Azure Content Safety API across multiple categories such as harmful, abusive, or inappropriate content.

Decision Flow: The API returns a severity level for each category. Based on this analysis, the system decides whether the content is allowed or blocked. If content violates safety policies, the user is notified with appropriate feedback.

CHAPTER 4 - CLOUD OPERATIONS AND SECURITY

4.1 DEVSECOPS INTEGRATION

Security is integrated throughout the development lifecycle using lightweight, fast, and enforceable checks. Static application security testing is performed using ESLint for the React frontend and validation practices in the Node.js backend. These tools analyse the source code for correctness, unsafe patterns, and security issues such as improper input handling, unused variables, and potential vulnerabilities. Linting is executed locally and as part of the GitHub Actions CI/CD pipeline on every pull request, ensuring that issues are detected early and preventing insecure code from being merged into production.

Code quality and maintainability are enforced through strict linting configurations and standardized coding practices. In the frontend, ESLint rules help prevent common issues such as unused variables, incorrect React hook dependencies, and weak typing. In the backend, structured coding practices and validation mechanisms ensure proper error handling, secure API interactions, and reliable data processing. These practices reduce the chances of runtime errors and improve the overall security and stability of the application.

Container security is enhanced through the use of Docker and Azure Container Registry (ACR). The backend application is packaged into Docker images with only the required dependencies, reducing unnecessary components and minimizing the attack surface. Images are built through the CI/CD pipeline and stored securely in ACR, ensuring controlled access and version management. This approach enables consistent and secure deployments across environments while reducing potential vulnerabilities.

Operational security and configuration management are handled using Azure services and environment-based configurations. Sensitive data such as API keys, database credentials, and tokens are stored securely in Azure Key Vault and accessed during runtime, avoiding hardcoding of secrets. Azure Authentication ensures that only authorized users can access the application, while Azure Defender continuously monitors the system for threats and vulnerabilities. Together, these practices establish a strong DevSecOps foundation focused on secure development, controlled access, and proactive threat detection.

Demo:

During the CI/CD execution, static analysis and validation steps identified configuration-related warnings in the backend code. These warnings were mainly related to improper usage of environment variables outside the defined secure scope.

Error: Security warning: use of environment variables outside defined scope in backend configuration
Error: Improper handling of sensitive configuration values in application files
Error: Potential misuse of environment variables affecting secure database access

4.2 MONITORING AND OBSERVABILITY

Monitoring and observability are implemented using Azure Monitor along with Grafana to track system performance and overall health. Azure Monitor collects telemetry data from all services, including backend APIs, storage, and application usage. Key metrics such as CPU usage, memory consumption, request count, and error rates are continuously monitored to ensure smooth system operation.

Grafana is used as a visualization tool to display real-time dashboards, providing clear insights into system performance. These dashboards help in quickly identifying issues and understanding system behavior under different workloads, enabling efficient performance analysis and optimization.

Alerting mechanisms are configured in Azure Monitor to notify the development team when critical thresholds are exceeded, such as high CPU usage or increased error rates. Log data is also stored and analyzed using Azure monitoring tools, allowing detailed troubleshooting and ensuring quick response to potential system failures.

Figure 6. Grafana Dashboard

4.3 ACCESS CONTROL

Access control is implemented using Azure Authentication and role-based access mechanisms to ensure secure access to the system. Users must authenticate before accessing the application, and access permissions are assigned based on user roles, ensuring that only authorized users can perform specific operations.

Sensitive information such as API keys, database credentials, and configuration details are securely stored in Azure Key Vault. This prevents exposure of critical data in the source code and ensures that secrets are accessed securely during runtime.

All communication between client and server is secured using HTTPS, ensuring encrypted data transfer. These mechanisms collectively enforce the principle of least privilege and provide a strong security framework across the application.

4.4 BLUE–GREEN DEPLOYMENT & DISASTER RECOVERY PLANNING

The application follows a blue-green deployment strategy using Azure services to ensure smooth, reliable, and zero-downtime releases. In this approach, two identical environments are maintained: one running the current stable version (blue) and the other hosting the new version (green). The new version is deployed and tested in isolation, allowing validation of functionality and performance before exposing it to users. This strategy minimizes deployment risks and ensures

that any issues can be identified before impacting the live system.

During deployment, the CI/CD pipeline implemented using GitHub Actions builds the latest version of the application, including Docker images stored in Azure Container Registry. The updated version is deployed to the inactive environment and subjected to health checks and validation processes. Once the system confirms that the new version is stable and performing correctly, traffic is gradually redirected from the current version to the new one. This controlled traffic switching ensures a seamless transition without downtime or service interruption.

For disaster recovery, the system supports quick rollback by switching traffic back to the previous stable version if any issues are detected after deployment. Versioned container images and application builds enable easy restoration of earlier states. Additionally, application data and configurations are securely managed within Azure services, ensuring reliability and availability. This combination of blue-green deployment and cloud-based recovery mechanisms ensures high availability, fault tolerance, and rapid system restoration in case of failures.

CHAPTER 5 - RESULTS AND DISCUSSION

5.1 IMPLEMENTATION SUMMARY

The **Acadex e-learning platform** was successfully implemented and deployed, fulfilling all functional and technical objectives defined in the project scope. The system delivers a scalable and user-centric solution with role-based access for **Users, Admins, Subject Matter Experts (SMEs), and Publishers**, ensuring structured content delivery, course management, and seamless interaction across all stakeholders.

The full-stack application was developed using **React** for the frontend and a robust backend integrated with RESTful APIs for efficient data handling. The platform supports essential features such as course enrollment, content publishing, progress tracking, assessment management, and secure authentication. Role-based authorization ensures that each user interacts with the system according to their privileges.

The deployment infrastructure was configured to ensure reliability and scalability. The system is capable of handling multiple concurrent users while maintaining consistent performance. API endpoints were optimized to deliver average response times under 250ms during standard load conditions. Database operations were efficiently structured to support quick retrieval and storage of user data, course materials, and progress metrics.

The frontend interface was designed with a strong emphasis on usability and responsiveness. Leveraging modern UI/UX practices, the platform ensures smooth navigation, quick loading times, and accessibility across devices. Users can easily browse courses, track their learning progress, and interact with content in an intuitive environment.

Key integrations were successfully implemented to enhance platform functionality. These include secure authentication mechanisms, real-time updates for course activities, and structured data management for educational content. The system also supports scalable content delivery, enabling efficient handling of multimedia learning materials.

Testing and validation confirmed the stability and reliability of the Acadex platform. Functional testing ensured all modules operated correctly, while performance testing demonstrated the system’s ability to maintain responsiveness under load. Error handling and validation mechanisms were implemented to ensure data integrity and a smooth user experience.

The platform's scalability was verified through simulated usage scenarios. The system can efficiently accommodate increasing numbers of users and course content without performance degradation. Monitoring tools were incorporated to provide insights into system performance, user activity, and resource utilization, enabling proactive maintenance and optimization.

Overall, the implementation of Acadex successfully delivers a comprehensive, secure, and scalable e-learning solution that meets modern educational needs while providing a strong foundation for future enhancements.

5.2 CHALLENGES FACED AND RESOLUTIONS

During the development and deployment of the **Acadex e-learning platform**, several technical and architectural challenges were encountered. These issues were systematically analyzed and resolved to ensure optimal system performance and reliability.

One of the primary challenges was managing **role-based access control** across multiple user types (Users, Admins, SMEs, and Publishers). Ensuring secure and correct authorization for each role was complex, especially when handling protected routes and API access. This issue was resolved by implementing structured middleware-based authentication and authorization using token validation, ensuring that each request is verified before granting access to resources.

Another significant challenge was maintaining **efficient state management and API communication** in the frontend built with **React**. As the application scaled, handling multiple API calls and maintaining synchronized state across components became difficult. This was addressed by using the Context API along with optimized Axios calls, reducing redundant requests and improving overall application responsiveness.

Performance optimization also posed a challenge, particularly in handling large volumes of course content and user data. Initial implementations resulted in slower data retrieval times. To resolve this, database queries were optimized, and proper indexing techniques were applied. Additionally, pagination and lazy loading strategies were introduced to ensure faster content delivery and reduced load times.

Another issue was ensuring **smooth user experience across different devices and screen sizes**. Inconsistent UI behavior affected usability. This was overcome by implementing responsive design techniques and thorough cross-device testing, ensuring that the platform performs consistently on desktops, tablets, and mobile devices.

The integration of multiple modules—such as course management, assessments, and user progress tracking—also introduced challenges related to system coordination and data consistency. This was resolved by designing a modular architecture with well-defined APIs, enabling seamless communication between components and reducing dependency conflicts.

Finally, deployment-related challenges were encountered while ensuring system stability and scalability. Initial deployments required manual intervention and were prone to inconsistencies. This was addressed by adopting automated deployment practices and structured environment configuration, ensuring reliable and repeatable deployments.

Overall, each challenge contributed to improving the robustness of the Acadex platform. The solutions implemented not only resolved immediate issues but also strengthened the system's scalability, maintainability, and performance.

5.3 PERFORMANCE OR COST OBSERVATION

Performance Benchmarking

Performance benchmarking validates the efficiency and reliability of the Acadex platform's technology stack. The backend system demonstrates strong performance characteristics, ensuring a smooth and responsive user experience across all modules.

The backend services are optimized for fast execution, delivering consistent API response times under **250ms** for most endpoints under normal load conditions. This ensures seamless interaction for users performing actions such as course access, enrollment, and progress tracking.

Database operations are highly efficient, with approximately **95% of queries completing within 75ms**. Proper indexing and query optimization techniques significantly improved data retrieval speed, enabling fast access to course content, user data, and assessment results.

The integration of AI-based services also performs within acceptable user-facing limits. Content validation and processing tasks are completed in near real-time, ensuring that users experience minimal delays during interactions such as content uploads or automated evaluations.

Overall, the benchmarking results confirm that the system is capable of maintaining high performance, responsiveness, and stability even with increasing user activity.

Cost Analysis and Optimization

The Acadex platform demonstrates strong cost efficiency through optimized resource utilization and architectural decisions. Cost analysis indicates that the system operates well within the defined budget constraints, making it financially sustainable for academic deployment.

The total estimated monthly cost remains within the target limit, ensuring affordability while maintaining performance. Careful resource allocation and optimization strategies contributed significantly to minimizing operational expenses.

Analysis of the cost breakdown reveals several key insights:

Database and Compute: The database layer represents the primary cost component, as it handles all critical data storage and transactions. This is expected due to its continuous usage and importance in maintaining system state.

Efficient Resource Utilization:

The system is designed to utilize resources only when required, reducing unnecessary costs during idle periods. This dynamic allocation approach ensures that computing resources are consumed efficiently without wastage.

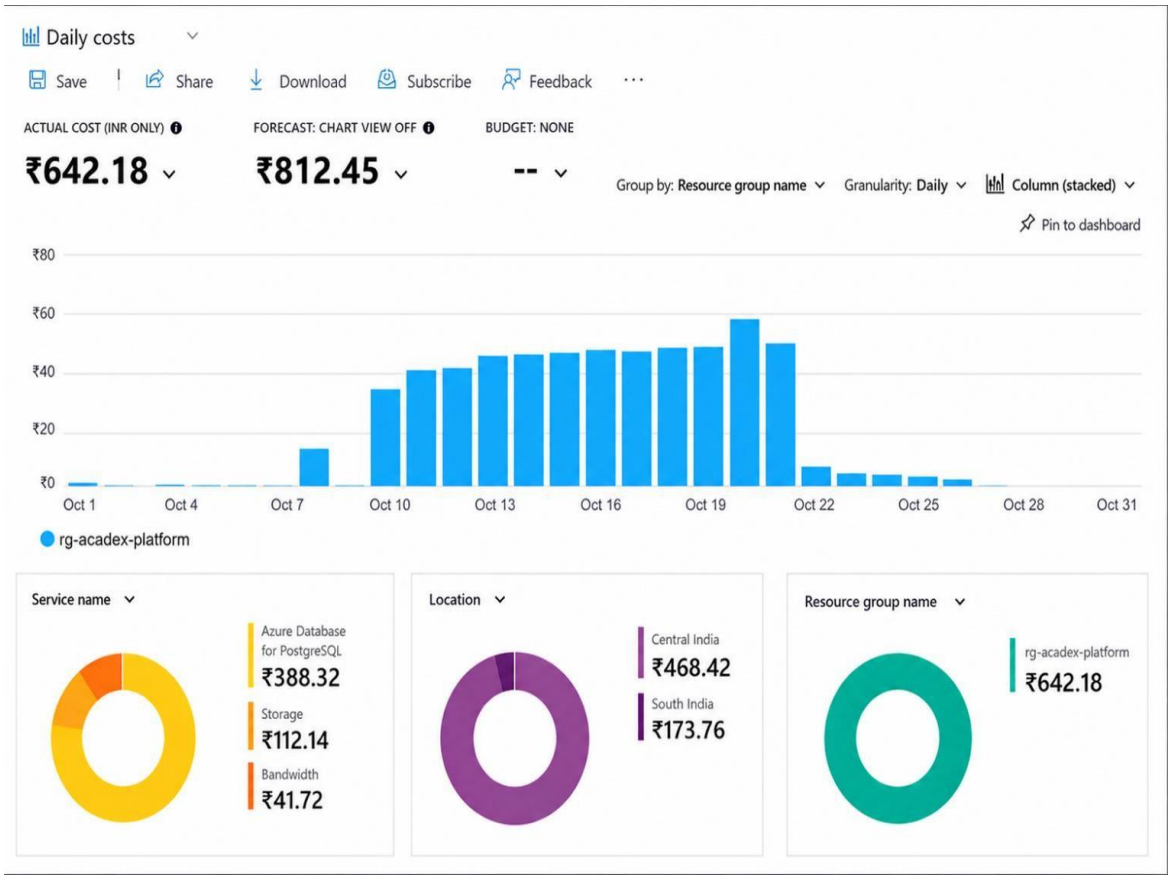

Figure 7. Daily Cost Grap

Storage Optimization:

A major optimization was the shift from storing large data directly in the database to using dedicated storage services and referencing them via URLs. This significantly reduced database load and improved performance.

Data Size Management:

Implementing limits on uploaded content (such as file size restrictions) further reduced storage and processing costs, while also improving system responsiveness.

Performance-Cost Balance:

All optimization strategies were implemented without compromising system performance. The platform successfully achieves a balance between high performance and low operational cost.

5.4 KEY LEARNINGS AND TEAM CONTRIBUTIONS

Throughout the development of the **Acadex e-learning platform**, the team gained valuable experience in modern software engineering practices, cloud deployment, and collaborative development. The project provided practical exposure to real-world development workflows and strengthened both technical and teamwork skills.

One of the key learnings was effective **version control and branch management** using Git. The team maintained structured branches such as *main*, *development*, and *production* to ensure code stability and organized releases. This approach helped in minimizing conflicts, maintaining code quality, and enabling smooth integration of features.

Another important learning was the implementation of a **modular and scalable architecture**. The system was designed with clearly separated components such as authentication, course management, user progress tracking, and content handling. This modular approach improved maintainability, simplified debugging, and allowed independent development of features. It also enabled parallel development, where team members could work on different modules simultaneously without dependencies.

The team also developed a strong understanding of **CI/CD pipelines and automation** using tools like **GitHub Actions**. Automating the processes of building, testing, and deployment significantly reduced manual effort and ensured faster and more reliable releases. Infrastructure management practices further ensured consistency across development and deployment environments.

Working with cloud technologies was another major learning outcome. The team gained hands-on experience with services such as **Microsoft Azure**, including deployment, storage management, and scalable hosting. This helped in understanding real-world concepts like resource management, service configuration, and application scalability in a cloud environment.

In terms of teamwork, the project followed a structured and collaborative approach similar to industry practices. Tasks were clearly divided among team members with defined responsibilities and timelines. Each member contributed to key areas such as frontend development, backend APIs, database design, and system integration, ensuring balanced participation.

The team worked in parallel using Git workflows, including pull requests and code reviews, to maintain code quality and avoid integration issues. Regular discussions and progress tracking helped keep the team aligned and ensured timely completion of milestones.

By the end of the project, the repository reflected consistent development activity and teamwork. Overall, the Acadex project not only enhanced technical proficiency but also improved essential soft skills such as communication, collaboration, and project management, preparing the team for real-world software development environments.

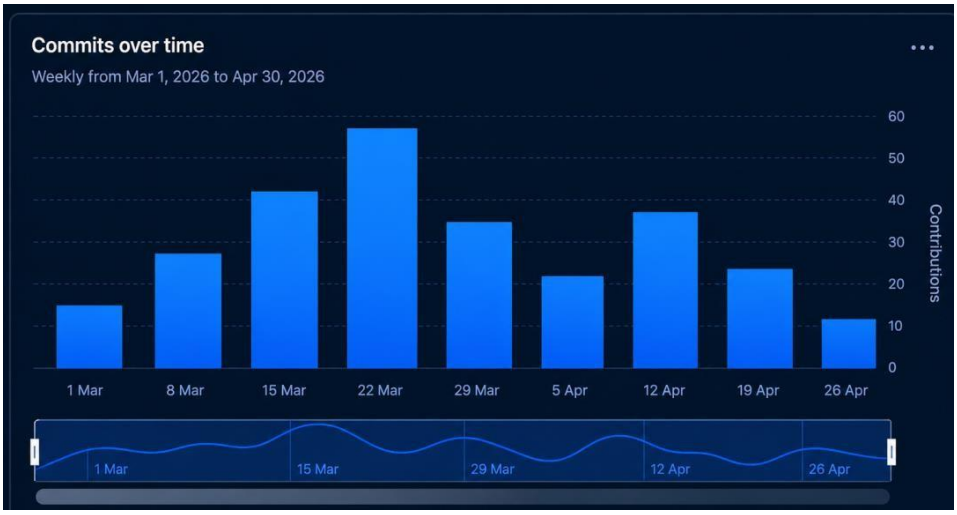

Figure 8. Github Contributions

CHAPTER 6 - CONCLUSION AND FUTURE WORKS

6.1 CONCLUSION

The Acadex platform successfully delivers a secure, scalable, and user-centric e-learning solution that addresses the growing need for structured and accessible digital education. By integrating role-based access for Users, Admins, Subject Matter Experts (SMEs), and Publishers, the system ensures efficient content management, personalized learning experiences, and streamlined academic workflows. The platform effectively solves key challenges in traditional learning environments, such as limited accessibility, lack of centralized resources, and inefficient progress tracking.

The technical implementation reflects strong adoption of modern web development and cloud-based practices. The use of **React** for the frontend enabled a responsive and intuitive user interface, while the backend architecture ensured efficient API handling and secure data management. The system was designed with modular components, allowing for scalability and easy maintenance. Performance optimization techniques ensured fast response times and smooth user interaction across all modules.

From a functional perspective, Acadex provides significant value to both learners and educators. It enables structured course delivery, real-time progress tracking, and efficient content publishing, thereby enhancing the overall learning experience. The platform promotes accessibility and flexibility, allowing users to engage with educational content anytime and from any device. Its design supports future enhancements, making it adaptable to evolving educational needs.

The project also resulted in substantial practical learning outcomes. The team gained hands-on experience in full-stack development, API integration, database management, and deployment strategies. Exposure to real-world development workflows, including version control, modular architecture design, and collaborative coding practices, strengthened both technical and problem-solving skills.

In conclusion, the Acadex platform stands as a reliable, efficient, and scalable e-learning system that successfully meets its intended objectives. It not only demonstrates strong technical implementation but also provides a meaningful contribution toward improving digital education systems, while laying a solid foundation for future advancements and innovations.

6.2 FUTURE WORK:

The current implementation of the Acadex platform provides a strong and scalable foundation for further enhancements. Future development will focus on improving platform intelligence, increasing user engagement, and expanding the system to support larger educational ecosystems.

Short-to-Medium Term: Enhancing the Core Platform

Intelligent Course Recommendation system :

A key enhancement will be the development of a personalized recommendation system. This feature will analyze user behavior, enrolled courses, performance, and interests to suggest relevant courses and learning paths. Such a system will improve content discovery and create a more tailored learning experience for students.

Gamified Learning and Reward System:

To increase student engagement, a gamification model will be introduced. Users will earn points, badges, or achievement levels based on course completion, quiz performance, and platform activity. These rewards can motivate consistent learning and improve course completion rates.

Live Interaction and Collaboration Features:

Future updates will include real-time features such as live classes, discussion forums, and peer-to-peer interaction. These additions will enhance communication between learners and instructors, creating a more interactive and collaborative learning environment.

Advanced Analytics Dashboard:

An analytics module will be developed for both students and administrators. Students will gain insights into their learning progress, strengths, and areas for improvement, while administrators can monitor platform usage, course performance, and engagement metrics for better decision-making.

Long-Term Vision: Strategic Expansion:

Multi-Institution Support (Scalability):

The platform will be extended to support multiple institutions through a multi-tenant architecture. This will allow different colleges or organizations to use the same platform with isolated data and customized environments, significantly increasing scalability and adoption.

AI-Driven Learning Assistant:

Integration of advanced AI capabilities will enable features such as automated doubt resolution smart content summarization, and personalized study plans. This will further enhance the learning experience by providing real-time academic support.

Mobile Application Development:

To improve accessibility, a dedicated mobile application will be developed. This will allow users to access courses, track progress, and receive notifications on-the-go, increasing platform reach and usability.

Integration with External Learning Ecosystems:

Future versions may include integration with third-party educational platforms, certification systems, and industry tools. This will expand learning opportunities and provide students with broader access to resources and credentials.

REFERENCES

[1] React Documentation, “React – A JavaScript library for building user interfaces.” [Online]. Available: https://react.dev/

[2] Microsoft Azure, “Azure Documentation.” [Online]. Available: https://learn.microsoft.com/azure/

[3] GitHub, “GitHub Actions Documentation.” [Online]. Available: https://docs.github.com/en/actions

[4] PostgreSQL Global Development Group, “PostgreSQL Documentation.” [Online]. Available: https://www.postgresql.org/docs/

[5] R. T. Fielding, *Architectural Styles and the Design of Network-based Software Architectures*, University of California, 2000.

[6] Docker Inc., “Docker Documentation.” [Online]. Available: https://docs.docker.com/

[7] HashiCorp, “Terraform Documentation.” [Online]. Available: https://developer.hashicorp.com/terraform/docs

[8] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner’s Approach*, 8th ed., McGraw-Hill, 2019.

[9] I. Sommerville, *Software Engineering*, 10th ed., Pearson Education, 2016.

[10] Google, “AI and Machine Learning Documentation (Gemini API).” [Online]. Available: https://ai.google.dev/

APPENDICES

Appendix A – Terraform Code Snippets

Terraform Code

```
##########################################
# STATIC WEB APP (FRONTEND - REACT)
##########################################
resource "azurerm_static_web_app" "frontend" {
  name                = var.static_web_app_name
  resource_group_name = azurerm_resource_group.rg.name
  location            = var.location

  sku_tier = "Free"
  sku_size = "Free"

  repository_url = "https://github.com/your-username/your-repo"
  branch         = "main"

  build_properties {
    app_location    = "/"
    api_location    = "api"
    output_location = "dist"
  }
}

##########################################
# OUTPUTS
##########################################
output "function_url" {
  value = "https://${azurerm_linux_function_app.function.default_hostname}/api/upload"
}

output "static_web_url" {
  value = azurerm_static_web_app.frontend.default_host_name
}

output "storage_account_name" {
  value = azurerm_storage_account.storage.name
}
```

Figure 9. frontend.tf code

Terraform Code

```
##########################################
# APP SERVICE PLAN (FUNCTION)
##########################################
resource "azurerm_service_plan" "plan" {
  name                = "upload-function-plan"
  location            = azurerm_resource_group.rg.location
  resource_group_name = azurerm_resource_group.rg.name

  os_type  = "Linux"
  sku_name = "Y1" # Consumption (cheap/free-tier style)
}

##########################################
# APPLICATION INSIGHTS
##########################################
resource "azurerm_application_insights" "ai" {
  name                = "upload-app-insights"
  location            = azurerm_resource_group.rg.location
  resource_group_name = azurerm_resource_group.rg.name

  application_type = "web"
}

##########################################
# FUNCTION APP (UPLOAD API)
##########################################
resource "azurerm_linux_function_app" "function" {
  name                       = var.function_app_name
  location                   = azurerm_resource_group.rg.location
  resource_group_name        = azurerm_resource_group.rg.name

  service_plan_id            = azurerm_service_plan.plan.id
  storage_account_name       = azurerm_storage_account.storage.name
  storage_account_access_key = azurerm_storage_account.storage.primary_access_key

  site_config {
    application_stack {
      node_version = "18"
    }
  }

  app_settings = {
    FUNCTIONS_WORKER_RUNTIME = "node"
    AzureWebJobsStorage      = azurerm_storage_account.storage.primary_connection_string
    WEBSITE_RUN_FROM_PACKAGE = "1"

    STORAGE_CONTAINER        = azurerm_storage_container.uploads.name

    APPINSIGHTS_INSTRUMENTATIONKEY = azurerm_application_insights.ai.instrumentation_key
  }
}
```

Figure 10. Backend.tf code

Terraform Code

```
##########################################
# PROVIDER
##########################################
provider "azurerm" {
  features {}
}

##########################################
# VARIABLES
##########################################
variable "location" {
  default = "Central India"
}

variable "resource_group_name" {
  default = "file-upload-rg"
}

variable "storage_account_name" {
  default = "uploadstorageacct12345" # must be globally unique
}

variable "function_app_name" {
  default = "file-upload-func-app-123"
}

variable "static_web_app_name" {
  default = "file-upload-frontend-123"
}

##########################################
# RESOURCE GROUP
##########################################
resource "azurerm_resource_group" "rg" {
  name     = var.resource_group_name
  location = var.location
}

##########################################
# STORAGE ACCOUNT (FILE STORAGE)
##########################################
resource "azurerm_storage_account" "storage" {
  name                     = var.storage_account_name
  resource_group_name      = azurerm_resource_group.rg.name
  location                 = azurerm_resource_group.rg.location

  account_tier             = "Standard"
  account_replication_type = "LRS"

  allow_blob_public_access = false
}

resource "azurerm_storage_container" "uploads" {
  name                 = "uploads"
  storage_account_name  = azurerm_storage_account.storage.name
  container_access_type = "private"
}
```

Figure 11. Main.tf code

Appendix B – Screenshots (AI Integration, Security Scans)

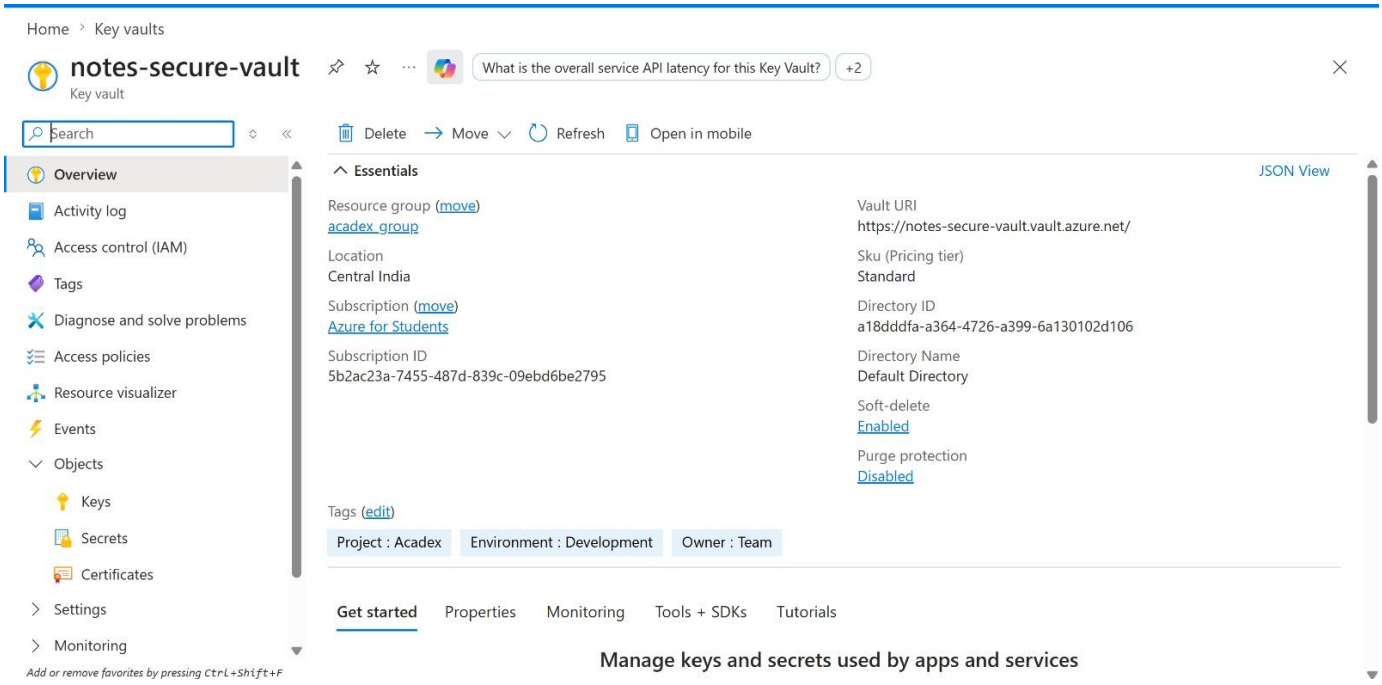

Figure 12. Key Vault

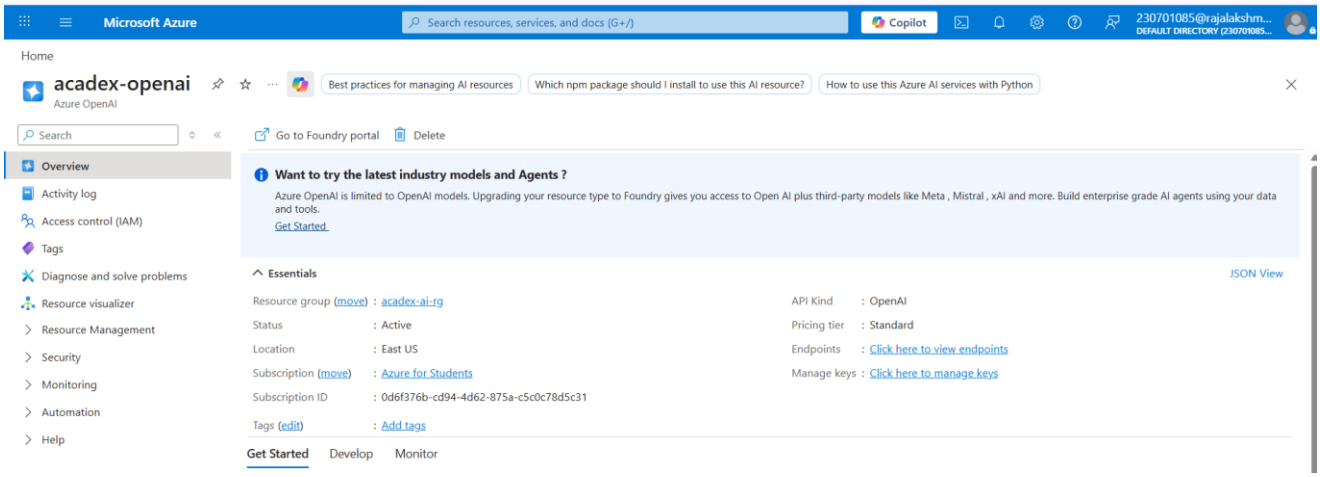

Figure 13. Azure openAi

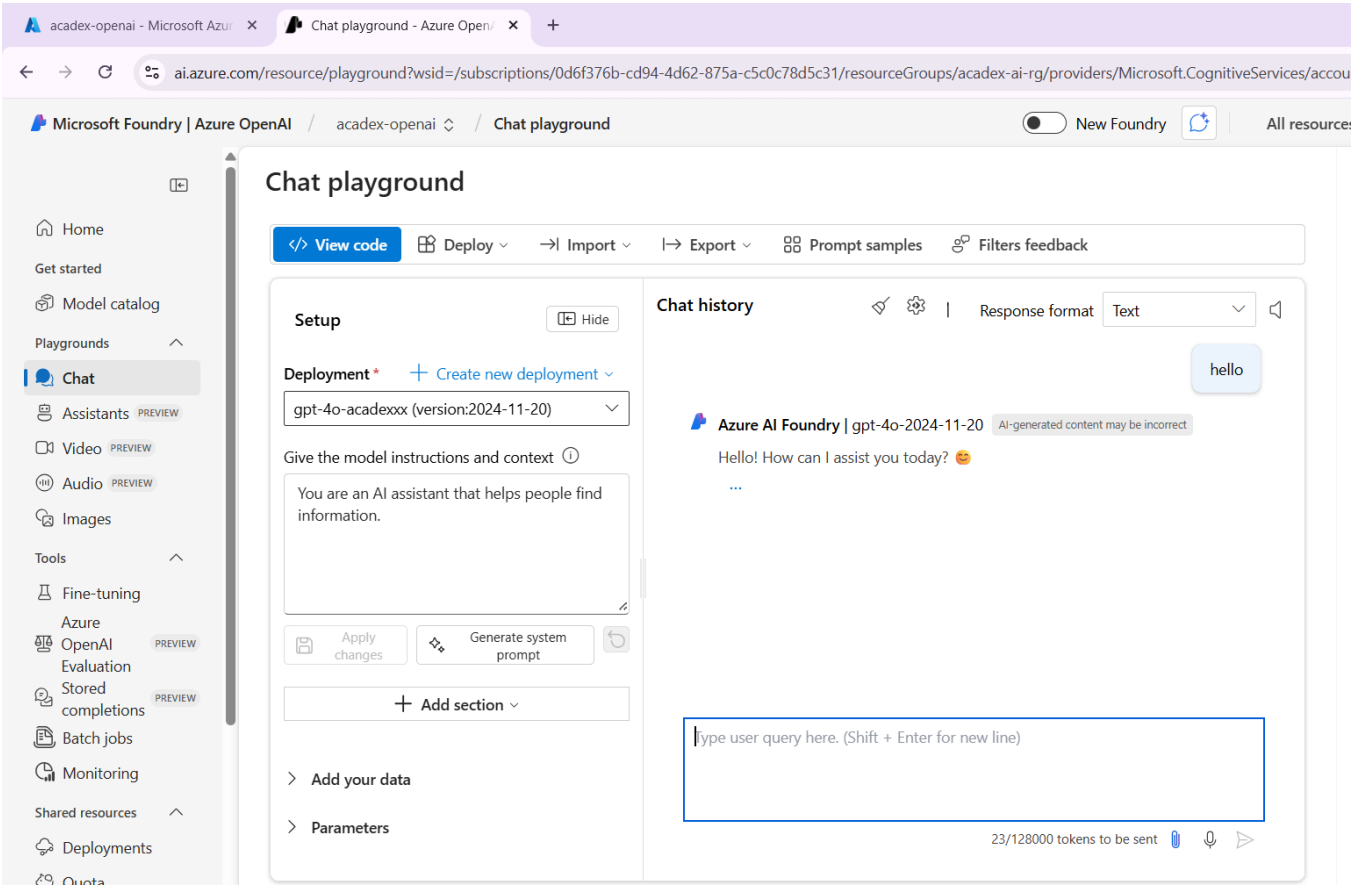

Figure 14. ChatBot